

Sitzung 1: Responsible RSE

Dr. Julian Dehne

Gesellschaft für Informatik e.V. (GI) · deRSE
julian.dehne@gi.de · ORCID 0000-0001-9265-9619

Einordnung

Andere verwenden die Ergebnisse von Forschungssoftware weiter. Daraus ergeben sich Anforderungen an:

- **Wiederverwendbarkeit:** Lizenz, Zitierbarkeit, Dokumentation
- **Integrität:** reproduzierbare Ergebnisse, nachvollziehbare Mathematik
- **Sicherheit:** Umgang mit Daten, Geheimnissen, Abhängigkeiten
- **Inklusion:** Barrierefreiheit, Sprache, Zugänglichkeit

Qualitätsdimensionen angelehnt an EVERSE RSQKit (EVERSE Consortium 2025) und FAIR4RS (Barker et al. 2022).

Nach dieser Einheit können Sie ...

- Qualitätskriterien für Forschungssoftware benennen und begründen
- ein Kriterium in eine prüfbare Regel übersetzen
- Querschnittsbelange mit Mitteln der aspektorientierten Programmierung vom Fachcode trennen
- die Grenzen automatischer Prüfungen reflektieren

Inputs

1. Organisatorisches
2. Aspects of Responsible Research Software Engineering and Project Introduction
3. Testing
4. Generierte Forschungssoftware
5. Green Coding
6. Open-Science-Qualitätsmerkmale
optional: Legal Aspects and Privacy
7. Nicht-deterministische Forschungssoftware (und LLMs)

Projektphase

8. Coding Dojo I
9. Coding Dojo II
10. Coding Dojo III

Abschluss

11. Studentische Präsentationen I
12. Studentische Präsentationen II
13. Studentische Präsentationen III
14. Studentische Präsentationen IV
15. Studentische Präsentationen V

Aufgabe

- Ein **Plugin** zum Open-Source-Projekt `rse_annotations` beitragen, das eine Verantwortungs-Gefahr (*responsibility hazard*) im Forschungscode erkennt.
- Das Plugin mit dem **Stand der Literatur** zu diesem Aspekt verknüpfen: Welche Kriterien gibt es, was lässt sich davon automatisch prüfen, was nicht?
- Seminararbeit und Vortrag dokumentieren beides.

Bewertung

Anteil	Leistung
30 %	Mitarbeit
50 %	Software + Seminararbeit
20 %	Vortrag

Punktesumme: Seminararbeit bis 15 Punkte, Software bis 12 Punkte. Die Punkte werden addiert; 15 Punkte sind die beste Note (Notenpunkte wie in der Oberstufe).

RSE-Grundlagen

Was ist Forschungssoftware?

- **Forschungssoftware:** Quellcode, Algorithmen, Skripte, Workflows und ausführbare Programme, die im Forschungsprozess oder für einen Forschungszweck entstanden sind (Gruenpeter et al. 2021).
- **Software in der Forschung:** wird für Forschung benutzt, ist aber nicht dafür entstanden, z. B. Betriebssysteme oder allgemeine Bibliotheken.
- Die Spannweite reicht vom einmaligen Auswertungsskript bis zur Infrastruktur, die eine ganze Fachgemeinschaft nutzt.

Forschungssoftware ist damit selbst ein Forschungsergebnis: Von ihr hängt ab, ob sich Ergebnisse nachvollziehen und wiederverwenden lassen.

Was ist Research Software Engineering?

- **Research Software Engineers (RSEs)** entwickeln Forschungssoftware und verbinden Softwaretechnik mit dem Verständnis für die Forschung, der sie dient.
- Der Begriff entstand 2012 auf einem Workshop des britischen Software Sustainability Institute (Hettrick 2016).
- Kompetenzen in drei Bereichen (Goth et al. 2024):
 - ▶ Softwaretechnik
 - ▶ Forschung
 - ▶ Kommunikation und Arbeit im Team

Vier Säulen, auf denen RSE aufbaut (Cohen et al. 2021):

- **Softwareentwicklung:** Forschungssoftware bauen und pflegen
- **Community:** Austausch über Fächer und Einrichtungen hinweg
- **Ausbildung:** Forschende und RSEs qualifizieren
- **Policy:** Anerkennung, Finanzierung, Karrierewege

Für Deutschland beschreibt ein Positionspapier aus der de-RSE-Community Stand, offene Probleme und Forderungen zu nachhaltiger Forschungssoftware (Anzt et al. 2021).

Aspekte

Manche Anforderungen betreffen viele Stellen im Code, gehören aber zu keiner davon, z. B. Logging, Zugriffsrechte, Transaktionen, Zeitmessung, Prüfung.

Ohne Trennung

- *Scattering*: derselbe Code an vielen Stellen verstreut
- *Tangling*: Fachlogik und Querschnittslogik in einer Funktion vermischt

Aspektororientierte Programmierung (AOP)

- **Join Point**: Stelle im Programmablauf, z. B. ein Funktionsaufruf
- **Pointcut**: Auswahl von Join Points
- **Advice**: Code, der an diesen Stellen ausgeführt wird
- **Aspekt**: Pointcut und Advice zusammen

Verantwortung im Code

Aspekte von Verantwortung in RSE

Im Code prüfbar

- Lizenzdatei vorhanden (Wiederverwendung)
- Zufalls-Seeds gesetzt (Vertrauenswürdigkeit)
- verschluckte Exceptions
- hart kodierte Konstanten
- Secrets im Repository (Schaden)
- Energieverbrauch messen (Nachhaltigkeit)

Durch Werkzeuge unterstützbar

- menschliche Inspektion generierten Codes
- Fairness-Tests auf Modellausgaben
- Barrierefreiheit von Abbildungen
- Offenlegung von LLM-Nutzung
- Protokoll der ausprobierten Modelle

Nur konzeptionell

- Dual-Use-Abwägung
- Zweckbindung von Daten
- Autorschaft und Anerkennung
- Wahl der Forschungsfrage

In dieser Informatik-Veranstaltung werden diese Aspekte nicht nur diskutiert, sondern als Prüfungen implementiert.

Themenwahl

- Testen von RSE
- Generierte Forschungssoftware
- Green Coding
- Open-Science-Qualitätsmerkmale
- Legal Aspects and Privacy
- Nicht-deterministische Forschungssoftware (und LLMs)
- Cybersicherheit
- AI-based research

- Anzt, Hartwig, Felix Bach, Stephan Druskat, et al. 2021. “An Environment for Sustainable Research Software in Germany and Beyond: Current State, Open Challenges, and Call for Action.” *F1000Research* 9: 295. <https://doi.org/10.12688/f1000research.23224.2>.
- Barker, Michelle, Neil P. Chue Hong, Daniel S. Katz, Anna-Lena Lamprecht, Carlos Martinez-Ortiz, Fotis Psomopoulos, Jennifer Harrow, et al. 2022. “Introducing the FAIR Principles for Research Software.” *Scientific Data* 9: 622. <https://doi.org/10.1038/s41597-022-01710-x>.
- Cohen, Jeremy, Daniel S. Katz, Michelle Barker, Neil Chue Hong, Robert Haines, and Caroline Jay. 2021. “The Four Pillars of Research Software Engineering.” *IEEE Software* 38 (1): 97–105. <https://doi.org/10.1109/MS.2020.2973362>.
- EVERSE Consortium. 2025. “RSQKit: Research Software Quality Kit.” <https://everse.software/RSQKit/>.
- Goth, Florian, Raniere Silva Alves, Michael Braun, et al. 2024. “Foundational Competencies and Responsibilities of a Research Software Engineer.” *F1000Research* 13: 1429. <https://doi.org/10.12688/f1000research.157778.1>.

- Gruenpeter, Morane, Daniel S. Katz, Anna-Lena Lamprecht, et al. 2021. "Defining Research Software: A Controversial Discussion." Zenodo. <https://doi.org/10.5281/zenodo.5504016>.
- Hettrick, Simon. 2016. "A Not-so-Brief History of Research Software Engineers." Software Sustainability Institute. <https://www.software.ac.uk/blog/2016-08-17-not-so-brief-history-research-software-engineers-0>.